BombLab 反汇编与 GDB 分析实验

计算机系统基础 Lab2 实验报告

杨悦一 24300220032

2026 年秋季学期

【摘要】本实验通过分析个人 bomb 的 x86-64 反汇编与静态数据，恢复六个正式关卡及 Honor 的输入约束和计算关系，并以 GDB 跟踪实际参数、寄存器和比较位置，将指令语义转换为通用 C 求解函数。六关分别涉及参数拆分与复合运算、数组递推、跳转表、二分递归、链表遍历及函数指针调用；Honor 则要求同时满足文本布局与上下文相关校验。实验以 seed 42 展示从具体 challenge 到口令的推导，再用公开列表以外的 seed 检验通用公式。终端截图显示正式六关公开测试为 60/60，Honor 公开测试为 10/10，非公开 seed 下六关及 Honor 均通过。本文记录关键汇编、数值演算、通用公式、调试问题及真实运行证据。

【关键词】x86-64；反汇编；GDB；无符号运算；递归；链表；函数指针

1 引言与实验要求

BombLab 要求从个人二进制恢复输入校验逻辑，并构造能够解除各关的口令。正式关卡入口为 phase_01 至 phase_06，求解接口为 solve_phase_1 至 solve_phase_6；每个接口接收 64 位 challenge，生成一行由空格分隔的无符号十进制整数。Honor 接口 solve_secret 还接收由前六行口令计算出的 context，输出完整第七行。[1]

本实验的分析顺序为“指令和静态数据—具体数值—通用公式—重新运行验证”。

2 实验环境与符号约定

2.1 环境与编译

实验在 Windows 的 WSL Ubuntu 环境中进行，目标格式为 elf64-x86-64。图 1 显示 Linux x86_64、GCC 13.3.0、GNU GDB 15.1 和 GNU objdump 2.42。make -B 使用 C11、-O2、-Wall、-Wextra、-Wpedantic、-Werror 编译 defuser，未出现编译错误。

```
yyy@yyy:~/test-git/ics-2026-bomblab$ cd ~/test-git/ics-2026-bomblab
mkdir -p images evidence
uname -sm
gcc --version | head -n 1
gdb --version | head -n 1
objdump --version | head -n 1
make -B
Linux x86_64
gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
GNU gdb (Ubuntu 15.1-1ubuntu1~24.04.1) 15.1
GNU objdump (GNU Binutils for Ubuntu) 2.42
gcc -I. -std=c11 -O2 -Wall -Wextra -Wpedantic -Werror defuser_main.c defuser.c tutor/phase0_solution.c ch
allenge.c cli.c -o defuser
```

图 1 实验环境及强制重新编译记录

2.2 数值宽度与基本指令

记 C 为 64 位 challenge，L=U32(C)，H=U32(C>>32)，U32 表示只保留低 32 位。除明确注明的 64 位移位和指针计算外，计算结果均按 uint32_t 的模 2^32 语义回绕。输出使用无符号十进制，例如 0xceaf10aa 对应 3467579562。

Linux x86-64 中，前六个整数或指针参数依次放在 rdi、rsi、rdx、rcx、r8、r9，32 位整数返回值通常在 eax。edi、esi、eax 是相应 64 位寄存器的低 32 位；写入这些 32 位寄存器会清零高 32 位。正式关卡入口的 rdi 指向输入字符串，rsi 是 challenge；进入辅助函数后必须重新解释参数。

表 1 本实验使用的关键指令语义

指令	解释及本实验用途
lea d(base,index,s),dst	计算 base+index*s+d，不读取该地址的内存
shr	无符号逻辑右移，高位补 0
rol 与 ror	循环左移与循环右移，移出的位绕回另一端
cmp src,dst	根据 dst-src 更新标志位，不保存差值
sete %al	相等时写 1，否则写 0
jb 与 jae	无符号的低于与高于或等于分支
call *地址表达式	通过表项中的函数指针进行间接调用

2.3 静态分析与动态验证

首先根据 valid_u32_fields 的第二个参数及 sscanf 返回值确定整数个数，再从最终 cmp 向前追踪期望值的来源。objdump -d 给出指令，objdump -s 给出节中实际字节。[2]

```
set pagination off
break *phase_01
run --seed 42 --phase 1
x/s $rdi
p/x $rsi
disassemble phase_01
```

GDB 的 => 指向下一条尚未执行的指令。si 可进入调用，ni 可跨过调用，advance *地址 前进到指定位置；读取寄存器前要确认当前 PC 和函数帧。set $C 等命令保存的是 GDB 便利变量，不修改被调试程序状态。[3]

3 正式六关的恢复与实现

3.1 第一关 参数拆分与复合运算

输入约束。 phase_01 要求一个整数。sscanf 将其保存到 rbp-0x14，随后调用 p1_compute(C)，将返回值 eax 与该输入比较，相等时 sete 写入成功标志。

```
40209d: mov  $0x1,%esi
4020aa: call valid_u32_fields
4020d8: mov  %r12,%rdi
4020db: call p1_compute
4020e0: cmp  %eax,-0x14(%rbp)
4020e3: sete %al
```

从汇编到公式。 p1_compute 将 C 拆成高 32 位 H、低 32 位 L，并保留 C>>16。它依次调用 step2、step0、step1。step2 的参数为 H 与 L；step0 的参数为 L 与 U32(C>>16)；step1 最后组合两次返回值。

```
# p1_step2
lea 0x1f33(%rsi),%eax
xor %edi,%eax
# p1_step0
lea (%rdi,%rdi,2),%eax
add %esi,%eax
xor $0x81b1,%eax
# p1_step1
lea (%rdi,%rsi,8),%eax
xor $0x8dac,%eax
```

第一段表示 (L+0x1f33)^H；第二段先计算 3*L，再加 U32(C>>16)，最后异或 0x81b1；第三段将第二段结果加上第一段结果的 8 倍，再异或 0x8dac。因此恢复的通式为：

```
uint32_t a = H ^ (L + 0x1f33u);
uint32_t b = (3u*L + (uint32_t)(C >> 16)) ^ 0x81b1u;
uint32_t answer = (b + 8u*a) ^ 0x8dacu;
```

数值演算。 seed 42 的 C=0x28efe333b266f103，L=0xb266f103，H=0x28efe333，U32(C>>16)=0xe333b266。先得 a=0x9a88f305；再得 b=0xfa6804de；组合后为 0xceaf10aa，即口令 3467579562。所有溢出均截断到低 32 位。

```
# 停在 call p1_compute 前
p/x $rdi
x/wu $rbp-0x14
disassemble p1_step2
disassemble p1_step0
disassemble p1_step1
ni
p/x $eax
p/u $eax
x/wu $rbp-0x14
```

```
林夏把旁边的椅子拖过来：“我看记录，你看程序。”
终端能读到接入请求，但还没有放行。旧的口令工具已经找不到了，
留下来的程序仍然在等一个合法的输入。

Phase 1 challenge: 0x28efe333b266f103
Input (unsigned decimal integers):
0

Breakpoint 1, 0x00000000004020db in phase_01 ()
(gdb) p/x $rdi
$1 = 0x28efe333b266f103
(gdb) x/wu $rbp-0x14
0x7fffffffcfec: 0
(gdb) disassemble p1_step2
Dump of assembler code for function p1_step2:
   0x0000000000402040 <+0>:     endbr64
   0x0000000000402044 <+4>:     lea    0x1f33(%rsi),%eax
   0x000000000040204a <+10>:    xor    %edi,%eax
   0x000000000040204c <+12>:    ret
End of assembler dump.
(gdb) disassemble p1_step0
Dump of assembler code for function p1_step0:
   0x0000000000402020 <+0>:     endbr64
   0x0000000000402024 <+4>:     lea    (%rdi,%rdi,2),%eax
   0x0000000000402027 <+7>:     add    %esi,%eax
   0x0000000000402029 <+9>:     xor    $0x81b1,%eax
   0x000000000040202e <+14>:    ret
End of assembler dump.
(gdb) disassemble p1_step1
Dump of assembler code for function p1_step1:
   0x0000000000402030 <+0>:     endbr64
   0x0000000000402034 <+4>:     lea    (%rdi,%rsi,8),%eax
   0x0000000000402037 <+7>:     xor    $0x8dac,%eax
   0x000000000040203c <+12>:    ret
End of assembler dump.
```

图 2 第一关参数及三个计算函数的反汇编

调试中的失败与修正。 图 3 中计算出的 eax 已是 3467579562，但栈中的输入仍是最初的占位值 0。因此 continue 后出现 BOOM。随后重新启动相同 seed 并手工输入该十进制口令，图 4 显示第一关通过。

```
(gdb) ni
0x00000000004020e0 in phase_01 ()
(gdb) p/x $eax
$2 = 0xceaf10aa
(gdb) p/u $eax
$3 = 3467579562
(gdb) x/wu $rbp-0x14
0x7fffffffcfec: 0
(gdb) x/2i $pc
=> 0x4020e0 <phase_01+80>:      cmp    %eax,-0x14(%rbp)
   0x4020e3 <phase_01+83>:      sete   %al
(gdb) continue
Continuing.
BOOM!!!
[BOOM] 本次维护中断。
城市恢复旧存档。终端里的记录还在，可以重新接入。

[Inferior 1 (process 24591) exited with code 01]
(gdb)
```

图 3 期望值已算出但输入仍为 0 导致校验失败



图 4 第一关输入 3467579562 后通过

3.2 第二关 六个整数的递推

确定首项。 valid_u32_fields 的参数为 6，sscanf 也必须返回 6，故答案有六项。首项计算依次执行右移 12 位、与 0xf 相与、乘 3，即 v[0]=3*((C>>12)&15)。

```
402160: shr $0xc,%rax
402164: and $0xf,%eax
402167: lea (%rax,%rax,2),%eax
40216a: cmp %eax,%esi
# 下标 i 从 1 开始，移位数从 9 开始
40216e: mov $0x9,%ecx
402173: mov $0x1,%edx
```

恢复递推。 循环开始时 esi 保存上一项，lea 得到 5*v[i-1]，sub 再减 i。另一条数据流将 C 右移 cl 位，取低 6 位并加 12；两条数据流异或后与当前输入项比较。每轮下标增加 1、移位数增加 3，所以第 i 轮的移位数为 6+3*i。

```
402178: mov %r13,%rax
40217b: lea (%rsi,%rsi,4),%esi
40217e: shr %cl,%rax
402181: sub %edx,%esi
402183: and $0x3f,%eax
402186: add $0xc,%eax
402189: xor %esi,%eax
40218b: mov -0x40(%rbp,%rdx,4),%esi
40218f: cmp %eax,%esi
402193: add $0x1,%rdx
402197: add $0x3,%ecx
```

```
v[0] = 3u * (uint32_t)((C >> 12) & 15u);
for (unsigned i=1; i<6; ++i)
    v[i] = (5u*v[i-1] - i)
         ^ ((uint32_t)((C >> (6u+3u*i)) & 63u) + 12u);
```

具体演算。 seed 42 的 C=0x47526757130f9f52，首项为 3*9=27。其余五项如下，计算表由上述通式代入得到；图 5 动态核对了 i=1 时 eax 与 esi 均为 157。

表 2 第二关 seed 42 的递推计算

i	移位数	5*v[i-1]-i	掩码项加 12	v[i]
1	9	134	27	157
2	12	783	69	842
3	15	4207	43	4164
4	18	20816	15	20831
5	21	104150	36	104178

```
# 在 phase_02 入口保存 C，或在循环比较处观察
set $C = (unsigned long long)$rsi
# 仅在入口执行上一行，不能在其他位置重取 rsi
break *0x40218f
# 重新运行并输入根据公式算出的六个整数
p/u $edx
p/u $eax
p/u $esi
```

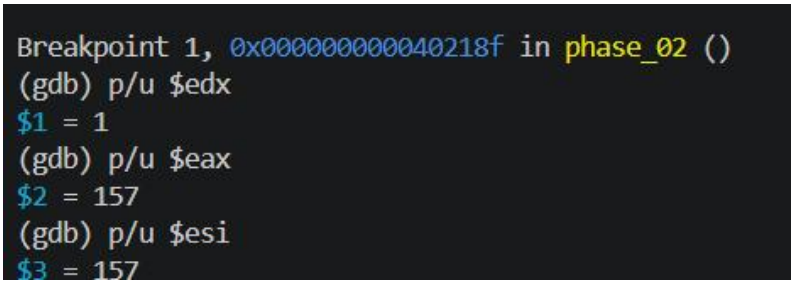


图 5 第二关下标 1 的计算值和输入值均为 157

口令为 27 157 842 4164 20831 104178，图 6 记录手工输入后通过。



图 6 第二关六项口令及通过记录

3.3 第三关 真实跳转表与分段函数

输入与参数。 第一项必须为 i=C&7；计算参数 x=U32(C>>3)^H。第二项来自 p3_dispatch(i,x) 的返回值。

```
402311: and $0x7,%edi
402314: cmp %edi,-0x18(%rbp)
40231f: shr $0x3,%rax
402323: shr $0x20,%rsi
402327: xor %eax,%esi
402329: call p3_dispatch
# dispatch 中按 8 字节表项间接跳转
40226d: notrack jmp *0x405820(,%rax,8)
```

表项恢复。 每个表项是一个小端 64 位地址，指向一个跳板，再由跳板转入实际 case 函数。索引不等于 case 名称中的编号。例如索引 4 的表项为 0x4022a8，该处跳到 p3_case_6，后者执行 (4*x)^0x2d60。

```
# $C 已在 phase_03 入口由 rsi 保存
set $i = (unsigned int)($C & 7)
set $x = (unsigned int)($C >> 3) ^ (unsigned int)($C >> 32)
x/8gx 0x405820
set $target = *(unsigned long long *)(0x405820 + 8*$i)
x/2i $target
disassemble p3_case_6
```

表 3 第三关全部索引对应的通用计算

i	跳板地址	case	F_i(x)
0	0x402288	1	5*x+0xa8b4
1	0x402290	0	5*x+0x1f6ad
2	0x402298	4	(x^0x79b7)+(x>>1)
3	0x4022a0	2	(x^0x3b4d)+(x>>2)
4	0x4022a8	6	(4*x)^0x2d60
5	0x4022b0	3	(4*x)^0x0f2d
6	0x402278	7	3*x+0xb69f
7	0x402280	5	5*x+0x27628

从当前数字到通式。 seed 42 时 C=0x581ce1ff0e4ae394，i=4，x=0xb9d5bd8d。U32(4*x)=0xe756f634，再异或 0x2d60 得 0xe756db54，即 3881229140。

```
uint32_t i = (uint32_t)C & 7u;
uint32_t x = (uint32_t)(C >> 3) ^ H;
/* 根据表 3 恢复 switch 的八个分支 */
/* 当前 i=4 的分支：y=(4u*x)^0x2d60u */
```



图 7 第三关索引 4 及计算结果通过校验

3.4 第四关 二分递归与返回值回代

参数来源。 入口仅要求一个整数。p4_walk 的参数依次为 target=C&127、low=0、high=127、salt=U32(C>>7)^H。函数计算 mid=low+((high-low)>>1)，再比较 target 与 mid。

```
402348: sub %esi,%edi
40234a: shr $1,%edi
40234c: add %esi,%edi
40234e: cmp %eax,%edi
402350: je  0x4023a0
402357: lea (%rdi,%rcx,1),%ebx
40235e: cmp %edi,%eax
402360: jb  0x402380
# 右分支返回：3*child ^ (mid+salt) ^ 2
# 左分支返回：3*child ^ (mid+salt) ^ 0x19
4023a0: lea 0x6a(%rdi),%eax
4023a3: xor %ecx,%eax
```

jb 是无符号“小于”。左分支把上界缩到 mid-1，右分支把下界改为 mid+1。目标总在 0..127 内，因此可到达命中中点的终止条件。

```
static uint32_t walk(uint32_t t, uint32_t lo,
                     uint32_t hi, uint32_t salt) {
    uint32_t mid = lo + ((hi-lo) >> 1);
    if (t == mid) return (mid+0x6au)^salt;
    if (t < mid)
        return 3u*walk(t,lo,mid-1u,salt) ^ (mid+salt) ^ 0x19u;
    return 3u*walk(t,mid+1u,hi,salt) ^ (mid+salt) ^ 2u;
}
```

路径与数值。 seed 42 的 C=0x09bc585a244823f2，target=114，salt=0xbdf4c81d。由区间和分支关系得到表 4，再从叶节点沿反方向逐层回代。

表 4 第四关 seed 42 的递归路径与返回值

区间	mid	方向	该层最终返回值
[0,127]	63	右	0x55c15e75
[64,127]	95	右	0xf811dcb9
[96,127]	111	右	0xc1f706ed
[112,127]	119	左	0x29569a21
[112,118]	115	左	0x318b70e4
[112,114]	113	右	0x842a92cf
[114,114]	114	命中	0xbdf4c8c1

例如叶节点先计算 (114+0x6a)^salt=0xbdf4c8c1，返回到 mid=113 的右分支后，计算 U32(3*r)^(113+salt)^2；依次回代到根得到 0x55c15e75，即 1438735989。

```
# 在 p4_walk 入口核对四个参数
info registers edi esi edx ecx
# 单步或在下一次 p4_walk 入口停下，观察区间缩小
bt
# 手工回代示例，$salt 已保存
set $r = (unsigned int)((114+0x6a)^$salt)
set $r = (unsigned int)((unsigned int)(3*$r) ^ (113+$salt) ^ 2)
p/x $r
```



图 8 第四关递归计算所得口令通过校验

3.5 第五关 链表布局与累积计算

从访存恢复节点。 输入有两个整数。起点来自 0x405b60 的八个指针；节点偏移 0 读取 32 位 value，偏移 4 读取 32 位 id，偏移 8 读取 64 位 next。因此节点布局为两个 uint32_t 和一个指针，共 16 字节。

```
402444: mov 0x405b60(,%rax,8),%rdx
402459: mov 0x4(%rdx),%esi
40245e: xor $0x51,%ah
402461: add (%rdx),%eax
402463: lea (%rax,%rax,4),%eax
402466: mov %esi,(%r8)
402469: xor %esi,%eax
402475: mov 0x8(%rdx),%rdx
```

ah 是 eax 的第 8 至 15 位，xor $0x51,%ah 等价于 eax^=0x5100。初值 acc=U32(C>>6)^H^0x5100。起点先被处理一次，循环再追加 (C>>3)&3 个节点，所以总数 n=1+((C>>3)&3)。每个节点执行 acc=U32(5*(acc+value))^id，并更新最后的 id。

表 5 第五关静态节点表

编号	地址	value	id	next 编号
0	0x405ba0	0xdef6	0x883	6
1	0x405bb0	0xa39e	0x720	9
2	0x405bc0	0xb730	0x92a	5
3	0x405bd0	0x3708	0x361	10
4	0x405be0	0x9d44	0x030	0
5	0x405bf0	0x4927	0x50c	7
6	0x405c00	0x3da2	0x681	1
7	0x405c10	0xc6e4	0x20e	11
8	0x405c20	0x7188	0x13a	NULL
9	0x405c30	0x61d6	0xb89	3
10	0x405c40	0x6a32	0xa7c	2
11	0x405c50	0xe91c	0x429	8

起点索引 0..7 对应节点编号 {4,0,6,1,9,3,10,2}。链表顺序由 next 决定，与节点在内存中的相邻顺序及 id 大小无关。

```
# $C 在 phase_05 入口保存
set $start = $C & 7
x/8gx 0x405b60
set $node = *(unsigned long long *)(0x405b60+8*$start)
x/2wx $node
x/gx $node+8
```

具体数值。 seed 42 的 C=0xde4431fa3c80db06，start=6，n=1，初值 acc=0x36b66396。表项指向节点 10，即 0x405c40；value=0x6a32，id=0xa7c。一次处理得到 0x11920e94，输出 2947846602684，图 9 记录通过。

```
node = starts[C & 7u];
n = 1u + ((C >> 3) & 3u);
acc = U32(C >> 6) ^ H ^ 0x5100u;
for (unsigned j=0; j<n; ++j) {
    acc = 5u*(acc + nodes[node].value) ^ nodes[node].id;
    last_id = nodes[node].id;
    node = nodes[node].next;
}
/* 输出 acc 和 last_id；acc 使用 uint32_t */
```



图 9 第五关累加值与末节点编号通过校验

3.6 第六关 函数指针的两次复合调用

恢复索引和顺序。 p6_dispatch 先计算 x=U32(C>>6)^H，再用 i=C&7 选择第一次函数。ebx=C&0x38 是第二个表项的字节偏移，除以 8 得 j=(C>>3)&7。第一次返回值异或 0x1222 后成为第二次输入，最后加 0x9393。

```
4025b0: and $0x38,%ebx
4025b3: and $0x7,%eax
4025b6: call *0x405e00(,%rax,8)
4025bd: xor $0x1222,%eax
4025c2: mov %eax,%edi
4025c4: call *0x405e00(%rbx)
4025cf: add $0x9393,%eax
```

表 6 第六关函数指针表及运算

索引	函数	F(x)
0	p6_op_0	ROL32(x,1)^0xf675
1	p6_op_5	U32(4*x+0xb38c)^U32(7*x)
2	p6_op_1	ROR32(x,7)
3	p6_op_4	U32((x^0x586c)+(x<<5))
4	p6_op_4	与索引 3 相同
5	p6_op_2	U32(x+0x3295)^(x>>3)
6	p6_op_0	与索引 0 相同
7	p6_op_3	U32((x^(x>>7))+0x17b9)

八个槽位只对应六种运算，表中重复指针必须保留。ROL32(x,1)=(x<<1)|(x>>31)，ROR32(x,7)=(x>>7)|(x<<25)，不能用普通移位代替。

```
# $C 已在入口保存
set $i = $C & 7
set $j = ($C >> 3) & 7
x/8gx 0x405e00
set $f1 = *(unsigned long long *)(0x405e00+8*$i)
set $f2 = *(unsigned long long *)(0x405e00+8*$j)
x/6i $f1
x/6i $f2
```

本例演算。 seed 42 的 C=0x37e9671c45376d5d，i=5、j=3，x=0x46fdbaa9。第一次实际调用 p6_op_2，得到 0x4e225a6b；异或 0x1222 得 0x4e224849。第二次调用 p6_op_4，得到 0x126b1945；再加 0x9393 得 0x126bacd8，即 309046488。

```
uint32_t x = (uint32_t)(C >> 6) ^ H;
x = operation(C & 7u, x) ^ 0x1222u;
x = operation((C >> 3) & 7u, x) + 0x9393u;
/* operation 的八个索引按表 6 实现 */
```



图 10 第六关两次复合运算所得口令通过校验

4 Honor 的进入与公式恢复

4.1 进入条件和输入准备

Honor 在不带 --phase 的完整运行中触发，要求前六行全部正确，并从文件或管道提供第七行。先把六关推导出的 seed 42 答案写入文件，末行使用 32 个 A、OPEN 与三个占位整数。前六行通过后，GDB 在 secret_gate 入口停下，图 11 证明实际进入该入口。

```
cat > honor-input.txt <<'EOF'
3467579562
27 157 842 4164 20831 104178
4 3881229140
1438735989
294784660 2684
309046488
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAOPEN0 0 0
EOF
gdb ./bomb

set pagination off
break *secret_gate
run --seed 42 < honor-input.txt
set $C = (unsigned long long)$rsi
set $K = (unsigned long long)$rdx
p/x $C
p/x $K
x/s $rdi
```

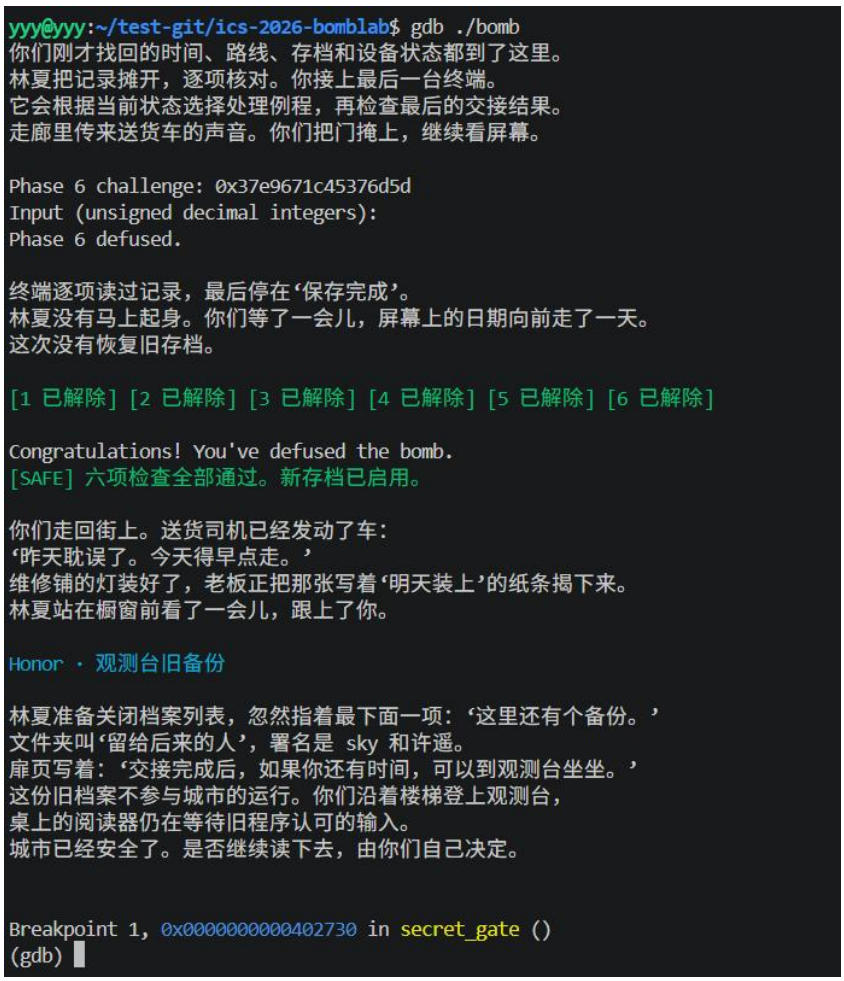


图 11 六关通过后停在 secret_gate 入口

4.2 从栈偏移恢复第七行布局

```
40274b: lea -0xd0(%rbp),%r12
40276c: call strlen
402771: sub $0x24,%rax
402775: cmp $0x7f,%rax
402779: jbe 0x402790
402796: call secret_copy
40279b: cmpl $0x4e45504f,-0xb0(%rbp)
4027ad: lea -0xac(%rbp),%rdi
4027b4: call secret_phase
```

length-36<=127 是无符号判断，故允许长度为 36..163。标记相对复制起点的偏移为 (-0xb0)-(-0xd0)=32；0x4e45504f 的小端字节 4f 50 45 4e 对应 OPEN。数字区位于偏移 (-0xac)-(-0xd0)=36。因此结构为“32 字节填充+OPEN+三个整数”。

4.3 从校验运算恢复三个整数

记 K 为 context。图 12 实测 C=0xccf635ee9e9e2fa4，K=0xee7b4a02d8251342，字符串中确有 32 个 A 和 OPEN0 0 0。K 由前六行经 transcript_context 得到，不是 seed，也不是固定常数；solve_secret 直接使用驱动传入的参数。

```
# secret_mix
ror $1,%eax
xor $0x1c80,%eax
# secret_phase 中的组合
imul $0x6872,%ecx,%r9d
shr $0x10,%r13
xor %r13d,%r9d
xor %r9d,%eax
shr $1,%esi
lea 0x74f2(%r8,%rcx,1),%edx
add %esi,%edx
xor %edx,%eax
```

第一项为 i=U32(K)&3。将 challenge 与 context 分别按高低 32 位异或得到 A 与 B；secret_mix 对 A^B 循环右移一位，再异或常量。之后加入索引乘积和 K>>16 得第二项 y；最后将 B、i、常量及 A>>1 的和与 y 异或，得第三项 z。

```
uint32_t i = (uint32_t)K & 3u;
uint32_t A = (uint32_t)C ^ (uint32_t)(C >> 32);
uint32_t B = (uint32_t)K ^ (uint32_t)(K >> 32);
uint32_t x = A ^ B;
uint32_t y = ((x >> 1) | (x << 31)) ^ 0x1c80u
             ^ (i*0x6872u) ^ (uint32_t)(K >> 16);
uint32_t z = y ^ (B+i+0x74f2u+(A >> 1));
```

本例 i=2，A=0x52681a4a，B=0x365e5940，x=0x6436430a；循环右移并异或后为 0x321b3d05，继续组合得 y=0x781935c4=2014918084，z=0x278bee9d=663482013。公式代入可用以下便利变量复算，不修改 bomb：

```
set $i = (unsigned int)($K & 3)
set $A = (unsigned int)$C ^ (unsigned int)($C >> 32)
set $B = (unsigned int)$K ^ (unsigned int)($K >> 32)
set $x = (unsigned int)($A ^ $B)
set $mix = (unsigned int)((($x>>1)|($x<<31)) ^ 0x1c80)
set $y = (unsigned int)($mix ^ ($i*0x6872) ^ (unsigned int)($K>>16))
set $z = (unsigned int)($y ^ (unsigned int)($B+$i+0x74f2+($A>>1)))
printf "%u %u %u\n", $i, $y, $z
```



图 12 Honor 参数读取及用计算结果替换第七行

4.4 重新运行验证

退出占位输入的调试会话，保留前六行并写入正确第七行，然后从文件重新运行。不能让未修改的 OPEN0 0 0 继续执行并期待通过。

```
head -n 6 honor-input.txt > honor-solved.txt
printf '%s\n' \
'AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAOPEN2 2014918084 663482013' \
>> honor-solved.txt
./bomb --seed 42 < honor-solved.txt
```

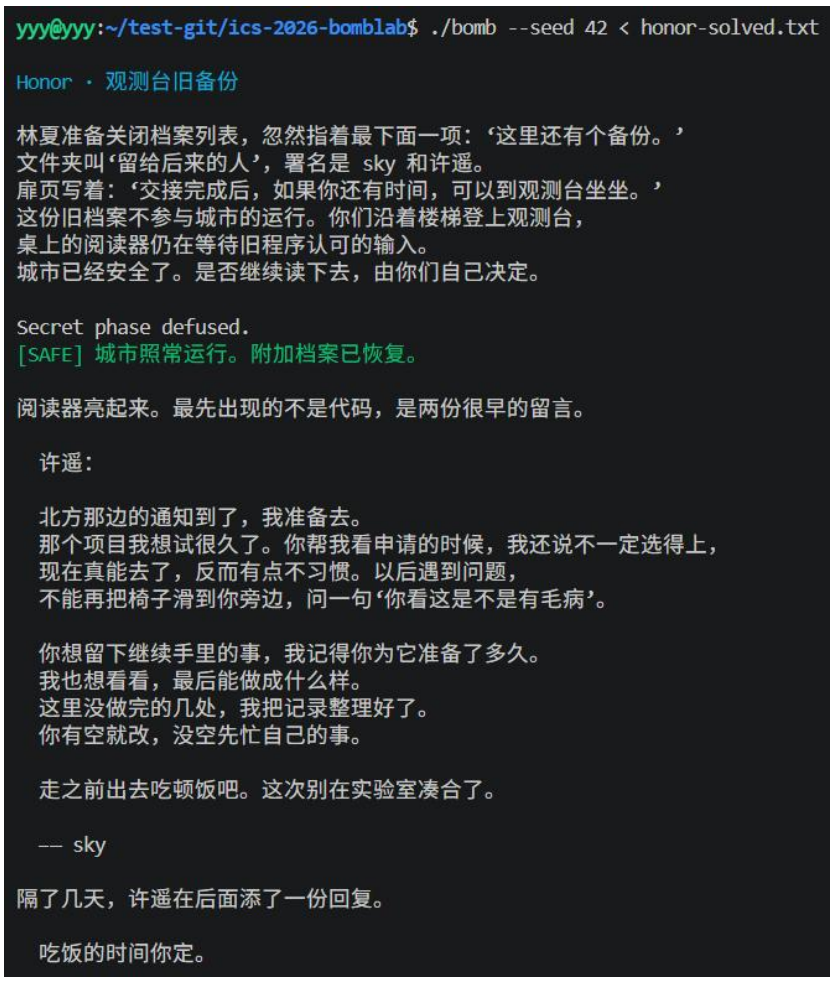


图 13 seed 42 下完整输入通过 Honor 校验

5 通用求解实现

5.1 接口与输出

实现保留原模板接口。正式六关分别计算 1、6、2、1、2、1 个 uint32_t 值，并用 write_answer 输出空格分隔的无符号十进制串。使用 PRIu32 匹配输出类型，成功返回 0，缓冲区不足或格式化失败时返回非零。walk 和 operation 是 static 辅助函数，分别表达递归和运算选择，不增加新的关卡接口。

write_answer 增加空指针、零容量与零字段数检查，不改变正常输出。Honor 使用 snprintf 写出前缀及三项十进制值。所有辅助日志与调试输出均不混入求解器 stdout，以免破坏一行口令的格式。

6 公开测试与非公开 seed 验证

6.1 公开测试来源与判定

public_test.py 明确从 tests/public.json 读取 cases，并让各正式关卡及 Honor 共用 cases["seeds"]。该次运行的十个 seed 为 0、1、7、42、99、123、2026、65535、987654321、18446744073709551615。非公开验证值 20261027 不在这个集合内。

```
cases = json.loads((root / "tests/public.json").read_text())
if args.secret:
    return 0 if check_secret(root, cases["seeds"]) else 1
phases = [args.phase] if args.phase is not None else cases["phases"]
for phase in phases:
    for seed in cases["seeds"]:
        # 生成答案，再交给同一 seed、同一 phase 的 bomb 检查
        ...
```

脚本会检查求解器返回状态、输出格式、超时和输出限制；正式关卡将答案交给 bomb。Honor 则先检查前六行，再拼接第七行，要求 bomb 正常结束且输出包含完整一行 Secret phase defused.。因此 Honor PASS 不只是检查字符串存在，还包含前六关的前置验证。

```
python3 public_test.py --phase 1
make test
```

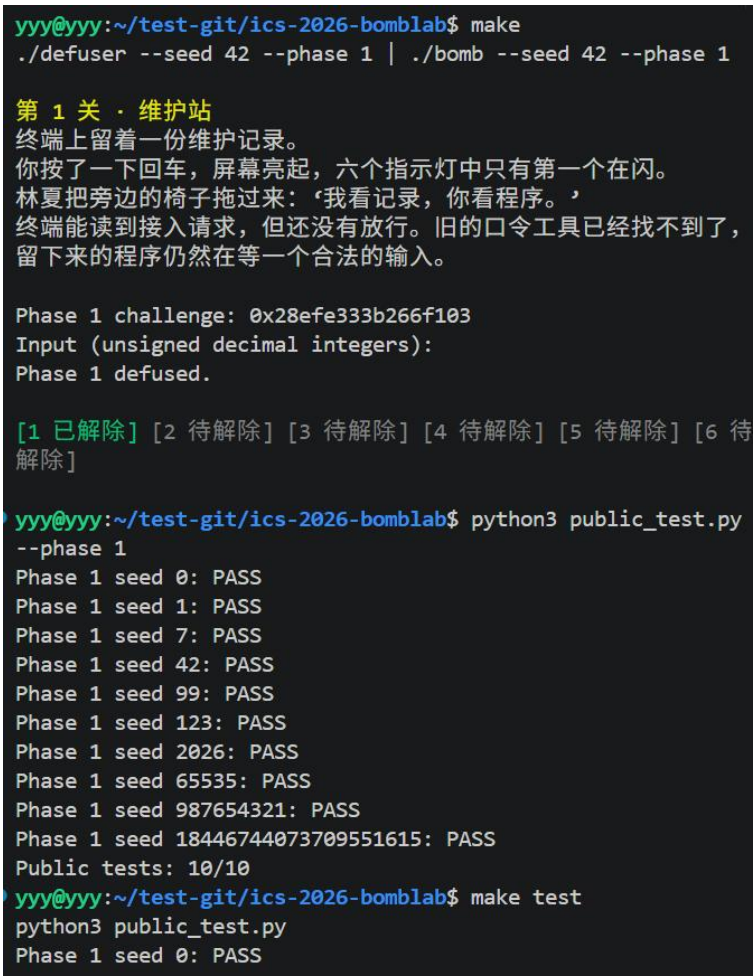


图 15 第一关公开列表及 10 次全部通过的记录

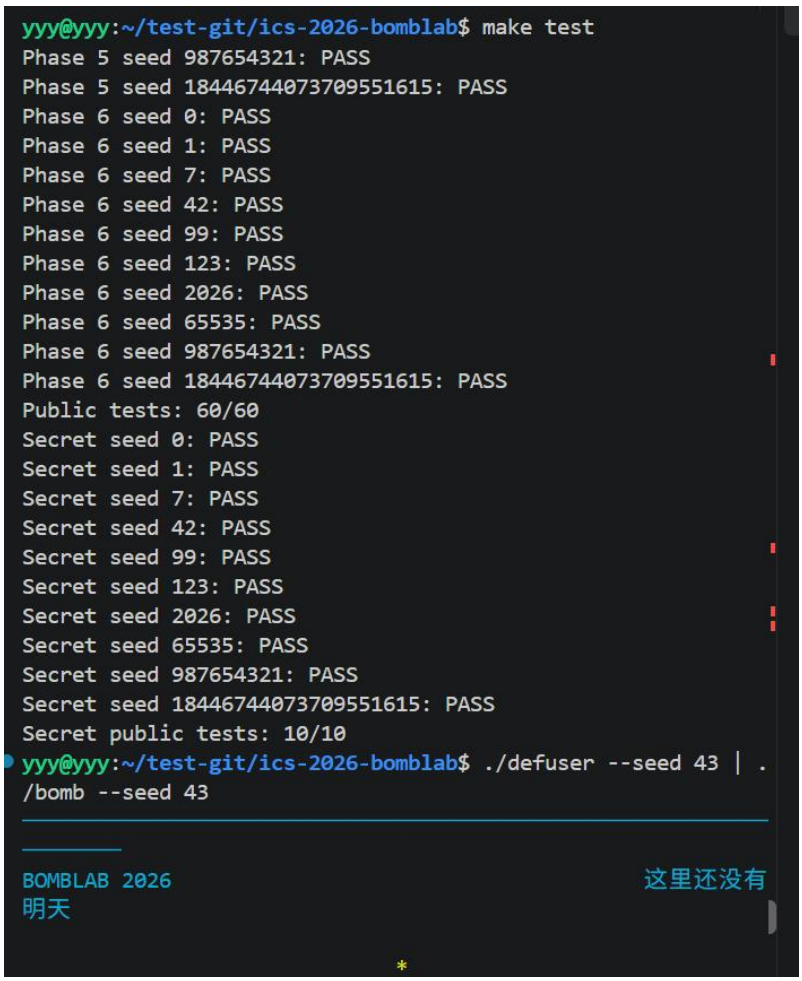


图 16 正式六关公开测试 60 比 60 与 Honor 10 比 10

图 16 底部显示启动 seed 43，该次也全部通过。

6.2 每个正式关卡的非公开 seed 记录

在相同 seed 下为每关生成答案，再将该行交给原始 bomb；终端同时输出关卡、seed、口令和 challenge，并保存日志。这里使用 defuser 是检查已经恢复的通用公式能否推广到新输入，不是用于替代前面的逆向推理。

```
S=20261027
for P in 1 2 3 4 5 6
do
    echo "===== Phase $P / seed $S ====="
    ./defuser --seed "$S" --phase "$P" \
        > "evidence/phase${P}-answer.txt" || break
    cat "evidence/phase${P}-answer.txt"
    ./bomb --seed "$S" --phase "$P" \
        < "evidence/phase${P}-answer.txt" \
        2>&1 | tee "evidence/phase${P}-seed${S}.log"
done
```

表 7 非公开 seed 20261027 的正式六关实测记录

关卡	challenge	实际口令	结果
1	0x635b6202ab529643	2756257510	通过
2	0xe0c8e9be2aab39d4	9 4 45 252 1246 6256	通过
3	0x9551bee1da18c95f	7 1717550362	通过
4	0xf8c6dde2f944ff26	599805517	通过
5	0xc64e3e27d20df988	2981259544 2179	通过
6	0xf0b8b1d373d19725	95424782	通过

```
yyy@yyy:~/test-git/ics-2026-bomblab$ for P in 1 2 3 4 5 6
do
    echo "===== Phase $P / seed $S ====="

    ./defuser --seed "$S" --phase "$P" \
        > "evidence/phase${P}-answer.txt" || break

    echo "计算出的口令: "
    cat "evidence/phase${P}-answer.txt"

    ./bomb --seed "$S" --phase "$P" \
        < "evidence/phase${P}-answer.txt" \
        2>&1 | tee "evidence/phase${P}-seed${S}.log"
done
===== Phase 1 / seed 20261027 =====
计算出的口令:
2756257510
Phase 1 challenge: 0x635b6202ab529643
Input (unsigned decimal integers):
Phase 1 defused.
===== Phase 2 / seed 20261027 =====
计算出的口令:
9 4 45 252 1246 6256
Phase 2 challenge: 0xe0c8e9be2aab39d4
Input (unsigned decimal integers):
Phase 2 defused.
===== Phase 3 / seed 20261027 =====
计算出的口令:
7 1717550362
Phase 3 challenge: 0x9551bee1da18c95f
Input (unsigned decimal integers):
Phase 3 defused.
===== Phase 4 / seed 20261027 =====
计算出的口令:
599805517
Phase 4 challenge: 0xf8c6dde2f944ff26
Input (unsigned decimal integers):
Phase 4 defused.
===== Phase 5 / seed 20261027 =====
计算出的口令:
2981259544 2179
Phase 5 challenge: 0xc64e3e27d20df988
Input (unsigned decimal integers):
Phase 5 defused.
===== Phase 6 / seed 20261027 =====
计算出的口令:
95424782
Phase 6 challenge: 0xf0b8b1d373d19725
Input (unsigned decimal integers):
Phase 6 defused.
```

图 17 seed 20261027 下逐关口令及六关全部通过的记录

6.3 Honor 的非公开 seed 记录

使用同一个 S=20261027 先生成前六行，再附加 --secret 输出。图 18 明确显示 seed、完整第七行、六关成功和 Secret phase defused.，满足 Honor 的前置条件和最终判断。

```
./defuser --seed "$S" > evidence/seven-lines.txt && \
./defuser --secret --seed "$S" >> evidence/seven-lines.txt
echo "===== Honor / seed $S ====="
tail -n 1 evidence/seven-lines.txt
./bomb --seed "$S" < evidence/seven-lines.txt \
    2>&1 | tee "evidence/honor-seed${S}.log"
```

该次第七行为 32 个 A、OPEN 和 3 1789546698 3445596646。

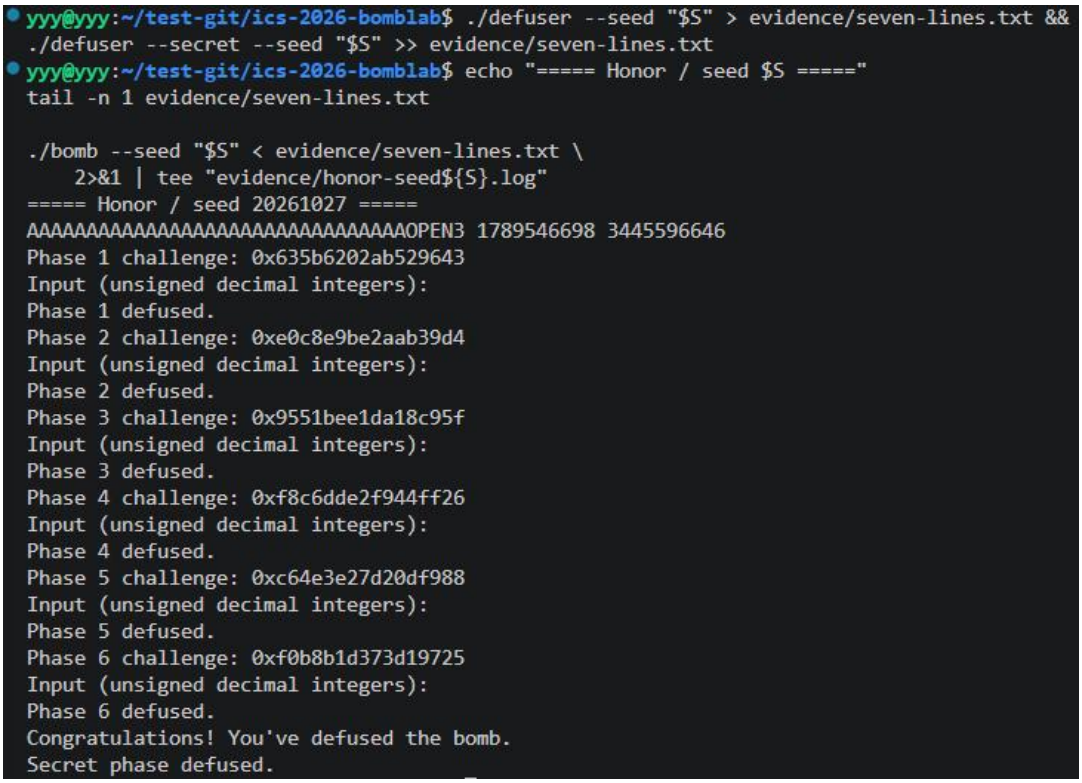


图 18 seed 20261027 下六关与 Honor 的完整通过记录

6.4 结果与覆盖范围

表 8 测试结果汇总

测试项目	结果	证据
seed 42 正式六关	6 关通过	图 4、6 至 10
seed 42 Honor	通过	图 11 至 13
正式六关公开测试	60/60	图 16
Honor 公开测试	10/10	图 16
非公开 seed 20261027 正式六关	6 关通过	表 7、图 17
非公开 seed 20261027 Honor	通过	图 18

实际上还测试了一些其他 seed，但在报告中仅呈现其中一例。调试过程篇幅较长，没怎么截图，很多图片是写报告重跑补的。

7 结论与辅助工具使用说明

本实验恢复并实现了正式六关及 Honor 的通用求解关系。分析中最重要的环节是先确定参数与位宽，再按实际控制流追踪数据；涉及间接跳转与链表时，同时恢复静态数据；涉及递归时区分下降路径和返回时的组合运算。现有运行证据显示，正式公开测试 60/60、Honor 公开测试 10/10，公开列表之外的 seed 20261027 下全部七关通过。实现不依赖固定答案表，而是根据 challenge 和 context 计算输出。

工具用途如下：GDB 用于断点、指令单步、寄存器和内存观察；objdump 用于导出指令和数据；GCC、Make 用于构建； public_test.py 用于公开测试；终端重定向和 tee 用于保存答案与验证日志。

AI 辅助使用披露：本次使用 ChatGPT/Codex 参与解释汇编、设计调试命令、进行辅助对照检查，并整理报告与排版。实际调试和运行截图来自本人终端操作。

参考资料

[1] 计算机系统基础课程．BombLab 实验说明．

[2] GNU Binutils．objdump 文档．https://sourceware.org/binutils/docs/binutils/objdump.html

[3] GNU GDB．Source and Machine Code．https://sourceware.org/gdb/current/onlinedocs/gdb.html/Machine-Code.html